

Apuntes para el desarrollo del proyecto

A modo de ejemplo, construiremos una aplicación académica para la carrera de Ingeniería Informática de INACAP utilizando:

Backend

- Node.js
- Express
- MongoDB Atlas
- Mongoose
- Dotenv
- Cors
- Nodemon

Frontend (más adelante)

- React
- Axios
- React Router
- Vercel

Despliegue

- Backend → Render
- Frontend → Vercel
- Base de datos → MongoDB Atlas

Etapas 1: Diseño del proyecto

Sistema Académico simplificado.

Tendremos tres colecciones principales, a modo de ejemplo podemos tener los siguientes datos:

Alumnos

```
{
  "_id": "...",
  "rut": "11111111-1",
  "nombre": "Juan",
  "apellido": "Pérez",
  "correo": "juan@inacapmail.cl",
  "semestre": 3
}
```

Docentes

```
{
  "_id": "...",
  "rut": "99999999-9",
  "nombre": "Carlos",
  "apellido": "Gómez",
  "correo": "carlos@inacap.cl",
  "especialidad": "Backend"
}
```

Asignaturas

```
{
  "_id": "...",
  "codigo": "PGY4121",
  "nombre": "Programación Backend",
  "semestre": 4,
  "docente": "ObjectId"
}
```

Relaciones

- Un docente puede impartir muchas asignaturas.
- Un alumno puede tener muchas asignaturas.
- Una asignatura puede tener muchos alumnos.

En MongoDB esto se implementará mediante referencias (ObjectId).

Conceptos claves para el manejo por código de los datos

Métodos HTTP

GET	Solicitar datos de documentos
POST	Insertar documento
PUT	Actualizar datos del documento
DELETE	Borrar documento

CRUD

Create	POST /api/alumnos
Read	GET /api/alumnos
Update	PUT /api/alumnos/:id
Delete	DELETE /api/alumnos/:id

MongoDB

Operaciones básicas:

```
find()  
findById()  
create()  
updateOne()  
deleteOne()
```

Arquitectura del backend

```
backend/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── models/  
│   │   ├── Alumno.js  
│   │   ├── Docente.js  
│   │   └── Asignatura.js  
│   ├── controllers/  
│   │   ├── alumnoController.js  
│   │   ├── docenteController.js  
│   │   └── asignaturaController.js  
│   ├── routes/  
│   │   ├── alumnoRoutes.js  
│   │   ├── docenteRoutes.js  
│   │   └── asignaturaRoutes.js  
│   ├── app.js  
│   └── server.js  
├── .env  
├── package.json  
└── .gitignore
```

Módulo 1

Fundamentos

- ¿Qué es una API?
 - Cliente y servidor.
 - HTTP.
 - JSON.
 - REST.
-

Módulo 2

Creación del proyecto Express

- Inicializar Node.
 - Instalar dependencias.
 - Crear servidor.
 - Middleware.
 - Variables de entorno.
-

Módulo 3

MongoDB Atlas

- Crear cuenta.
 - Crear Cluster.
 - Usuario.
 - Network Access.
 - String de conexión.
 - Conectar Express con Atlas.
-

Módulo 4

Modelado con Mongoose: Schema, Model, Validaciones.

Creación de: Alumno, Docente y Asignatura

Módulo 5

CRUD de Alumnos

- GET
- GET BY ID
- POST
- PUT
- DELETE

Pruebas con Postman.

Módulo 6

CRUD de Docentes

Módulo 7

CRUD de Asignaturas

Incluyendo: `populate()` para mostrar información del docente.

Módulo 8

Consultas avanzadas

Populate

```
Asignatura.find()  
.populate("docente")
```

Aggregate

Ejemplo:

Cantidad de alumnos por semestre.

Lookup

Simulación de JOIN.

```
{
  $lookup: {
    from: "docentes",
    localField: "docente",
    foreignField: "_id",
    as: "informacionDocente"
  }
}
```

Módulo 9

Deploy Backend

Render.

- Variables de entorno.
 - Build.
 - Start command.
 - Conexión Atlas.
-

Módulo 10

Frontend React

- CRUD visual.
 - Axios.
 - Formularios.
 - Consumo de API.
-

Módulo 11

Deploy Frontend

Vercel.

- Variables de entorno.
- URL del backend.
- Producción.

Recomendación pedagógica

La progresión ideal es:

1. Crear servidor Express.
2. Crear rutas de prueba.
3. Probar HTTP con Postman.
4. Conectar Atlas.
5. Crear primer modelo (Alumno).
6. Crear CRUD completo de Alumno.
7. Repetir patrón para Docente.
8. Repetir patrón para Asignatura.
9. Enseñar relaciones (`ObjectId`).
10. Enseñar `populate()`.
11. Enseñar `aggregate()`.
12. Deploy en Render.
13. Construcción del frontend.
14. Deploy en Vercel.

Esta estructura permite que los estudiantes entiendan primero HTTP y CRUD antes de introducir relaciones y consultas avanzadas de MongoDB.

MÓDULO 1: Fundamentos de APIs REST con Node.js

Objetivos de aprendizaje

Al finalizar este módulo, será capaz de:

- Comprender qué es una API.
- Entender la arquitectura Cliente-Servidor.
- Diferenciar Frontend y Backend.
- Comprender cómo funciona HTTP.
- Identificar los principales métodos HTTP.
- Entender el formato JSON.
- Comprender el concepto CRUD.
- Entender qué es una API REST.

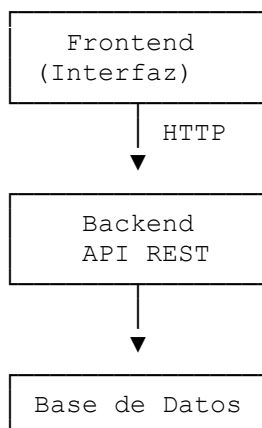
1. ¿Qué es una aplicación web moderna?

Cuando utilizamos aplicaciones como:

- Gmail
- Netflix
- Spotify
- Instagram
- Portal Académico INACAP

realmente estamos utilizando varios sistemas trabajando en conjunto.

Generalmente una aplicación moderna está compuesta por:



Frontend

Es la parte visual.

Lo que el usuario ve y utiliza.

Ejemplos:

- Botones
- Formularios
- Tablas
- Menús
- Gráficos

Tecnologías comunes:

- HTML
 - CSS
 - JavaScript
 - React
-

Backend

Es la lógica del sistema.

Se encarga de:

- Procesar solicitudes.
- Validar información.
- Aplicar reglas de negocio.
- Conectarse a la base de datos.
- Enviar respuestas al frontend.

Tecnologías comunes:

- Node.js
 - Express
 - Java
 - Python
 - .NET
-

Base de Datos

Es donde se almacenan los datos.

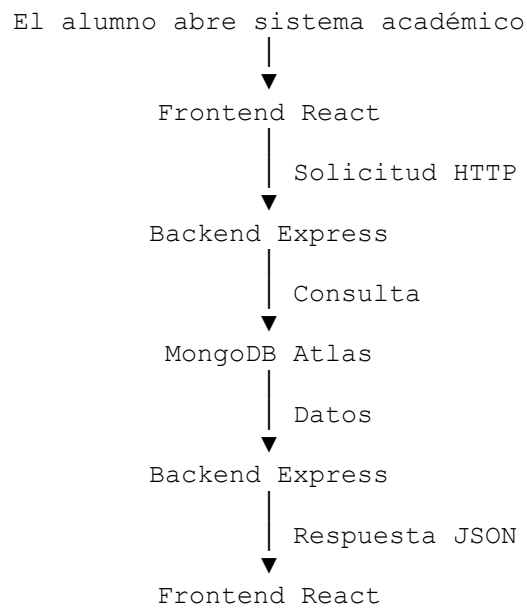
Ejemplos:

- Usuarios
- Productos
- Ventas
- Alumnos
- Docentes
- Asignaturas

En este proyecto utilizaremos **MongoDB Atlas**

2. Arquitectura Cliente-Servidor

Toda comunicación ocurre mediante solicitudes y respuestas. Por ejemplo:



Ejemplo real

El usuario presiona:

Ver listado de alumnos

El frontend envía:

GET /api/alumnos

El backend consulta MongoDB y responde:

```
[
  {
    "nombre": "Juan",
    "apellido": "Pérez"
  },
  {
    "nombre": "María",
    "apellido": "González"
  }
]
```

El frontend recibe esos datos y los muestra en pantalla.

3. ¿Qué es una API?

API significa: Application Programming Interface

Es un intermediario entre aplicaciones. Permite que dos sistemas se comuniquen.

Ejemplo cotidiano

Imaginemos un restaurante.

Cliente → Mesero → Cocina

El cliente no entra a la cocina. Habla con el mesero. El mesero lleva la solicitud. La cocina prepara la respuesta.

La API cumple el rol del mesero.

Frontend → API REST → Base de Datos

4. ¿Qué es HTTP?

HTTP significa: HyperText Transfer Protocol

Es el protocolo utilizado para intercambiar información entre cliente y servidor.

Cada vez que abrimos una página web estamos utilizando HTTP.

Solicitud HTTP

Una solicitud contiene:

- Método
- URL
- Headers
- Body (opcional)

Ejemplo: `GET /api/alumnos`

Respuesta HTTP

El servidor responde:

- Código de estado
- Headers
- Datos

Ejemplo: `200 OK`

5. Métodos HTTP

Los métodos HTTP indican qué acción se desea realizar.

GET

Obtiene información.

Ejemplo: `GET /api/alumnos`

Significa: `Obtén todos los alumnos.`

POST

Crea información.

Ejemplo: `POST /api/alumnos`
Body: `{ "nombre": "Juan", "apellido": "Pérez" }`

Significa: `Crear un nuevo alumno.`

PUT

Actualiza completamente un recurso.

Ejemplo: `PUT /api/alumnos/123`

Significa: `Actualizar alumno 123.`

PATCH

Actualiza parcialmente un recurso.

Ejemplo: `PATCH /api/alumnos/123`
`Body: { "semestre": 4 }`

Significa: `Actualizar solo el semestre.`

DELETE

Elimina información.

Ejemplo: `DELETE /api/alumnos/123`

Significa: `Eliminar alumno 123.`

6. Códigos de estado HTTP

Cuando una API responde, informa el resultado mediante códigos.

200 OK	Todo salió correctamente.
201 Created	Registro creado exitosamente.
400 Bad Request	Datos inválidos.
404 Not Found	Recurso no encontrado.
500 Internal Server Error	Error interno del servidor.

Diseño inicial de nuestra API

Vamos a construir las siguientes rutas:

Alumnos

```
GET      /api/alumnos
GET      /api/alumnos/:id
POST     /api/alumnos
PUT      /api/alumnos/:id
DELETE   /api/alumnos/:id
```

Docentes

```
GET      /api/docentes
GET      /api/docentes/:id
POST     /api/docentes
PUT      /api/docentes/:id
DELETE   /api/docentes/:id
```

Asignaturas

```
GET      /api/asignaturas
GET      /api/asignaturas/:id
POST     /api/asignaturas
PUT      /api/asignaturas/:id
DELETE   /api/asignaturas/:id
```

MÓDULO 2: Creación del Proyecto Backend con Node.js y Express

Objetivos de aprendizaje

- Instalar Node.js.
 - Crear un proyecto Node.js.
 - Instalar dependencias.
 - Crear un servidor Express.
 - Utilizar middleware.
 - Crear rutas básicas.
 - Levantar el servidor local.
 - Utilizar variables de entorno.
 - Preparar la estructura para conectar MongoDB Atlas.
-

1. Instalación de Node.js

Descargar desde: [Node.js Official Website](https://nodejs.org/es/) Se recomienda la versión LTS.

2. ¿Qué es npm?

npm significa: Node Package Manager

Es el gestor de paquetes de Node.js. Permite instalar librerías desarrolladas por la comunidad.

3. Creación del proyecto

Crear carpeta: `mkdir sistema-academico-api`

Ingresar: `cd sistema-academico-api`

Inicializar proyecto: `npm init -y`

Node creó automáticamente: `package.json`

4. Instalación de Express

Express es el framework más utilizado para crear APIs con Node.js.

Instalación: `npm install express`

5. Instalación de herramientas necesarias

Durante el curso utilizaremos: `npm install mongoose cors dotenv`

Instalar dependencia de desarrollo: `npm install -D nodemon`

¿Para qué sirve cada una?

mongoose	Conectar MongoDB con Node.js.
cors	Permitir comunicación con React.
dotenv	Leer variables de entorno.
nodemon	Reiniciar servidor automáticamente.

6. Estructura profesional del proyecto

Crear manualmente:

```
sistema-academico-api/  
├── src/  
│   ├── config/  
│   ├── controllers/  
│   ├── models/  
│   ├── routes/  
│   ├── middlewares/  
│   ├── app.js  
│   └── server.js  
├── .env  
├── .gitignore  
├── package.json  
└── package-lock.json
```

7. Configurar .gitignore

Crear:

.gitignore

Contenido:

```
node_modules
.env
```

No debemos subir a Github:

Dependencias , Contraseñas , Cadenas de conexión

8. Ajustes para utilizar JavaScript moderno (ES6/ES7+)

debemos modificar package.json. Agregar:

```
{
  "name": "sistema-academico-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  }
}
```

9. Crear archivo app.js

Ruta:

src/app.js

Contenido:

```
import express from "express";
const app = express();
export default app;
```

10. Crear archivo server.js

Ruta:

src/server.js

Contenido:

```
import app from "./app.js";
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor ejecutándose en puerto ${PORT}`);
});
```

Con ES Modules debemos especificar la extensión: `import app from "./app.js";`

No funciona: `import app from "./app";` Node.js requiere la extensión explícita.

11. Configurar Variables de Entorno

Crear archivo: `.env`
Contenido: `PORT=3000`

server.js

```
import dotenv from "dotenv";
import app from "./app.js";
dotenv.config();
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor ejecutándose en puerto ${PORT}`);
});
```

12. Primera Ruta

app.js

```
import express from "express";
const app = express();
app.get("/", (req, res) => {
  res.send("API Sistema Académico funcionando");
});
export default app;
```

13. Middleware JSON

app.js

```
import express from "express";
const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("API Sistema Académico funcionando");
});
export default app;
```

14. Ruta POST de prueba

app.js

```
import express from "express";
const app = express();
app.use(express.json());

app.get("/", (req, res) => {
  res.send("API Sistema Académico funcionando");
});

app.post("/prueba", (req, res) => {
  console.log(req.body);
  res.json({
    mensaje: "Datos recibidos",
    datos: req.body
  });
});

export default app;
```

15. Ejecutar servidor

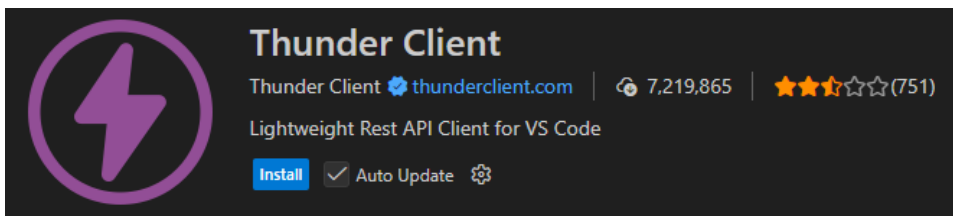
Terminal: `npm run dev`

Resultado esperado: Servidor ejecutándose en puerto 3000

Abrir navegador: `http://localhost:3000`

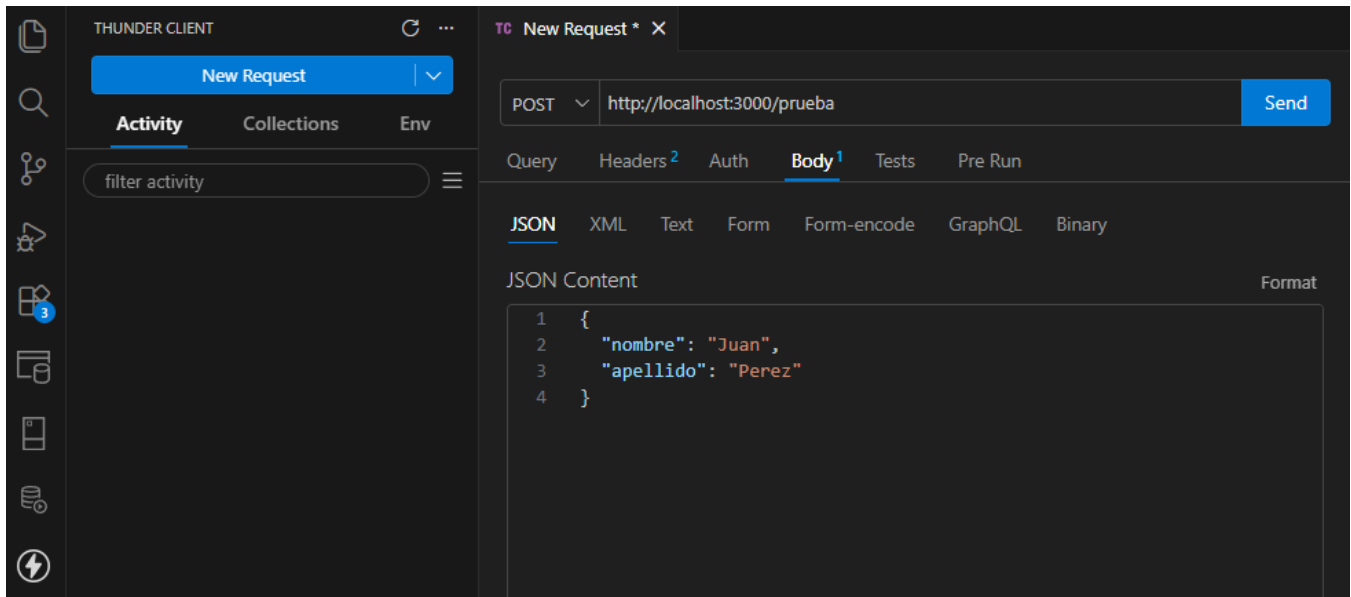
Respuesta: API Sistema Académico funcionando

16. Probando los métodos HTML



Instalamos en VSCode el complemento Thuder Client (o si prefiere PostMan) para hacer las solicitudes al servidor AP

Para probar que el servidor está recibiendo documentos a través del método POST, que nos permitirá insertar posteriormente los datos a una base de datos, hacemos un nuevo requerimiento (New Request)



Respuesta:

```
{
  "mensaje": "Datos recibidos",
  "datos": {
    "nombre": "Juan",
    "apellido": "Perez"
  }
}
```

Estructura actual recomendada

```
sistema-academico-api/
├── src/
│   ├── app.js
│   └── server.js
├── .env
├── .gitignore
├── package.json
└── package-lock.json
```

Actividad práctica

Implementar las siguientes rutas:

```
GET /
GET /saludo
GET /alumnos
GET /docentes
GET /asignaturas
```

Cada una debe devolver un mensaje JSON distinto.

Ejemplo:

```
app.get("/alumnos", (req, res) => {  
  res.json({  
    mensaje: "Listado de alumnos"  
  });  
});
```

Adelantando una buena práctica profesional

Como utilizaremos controladores, modelos y rutas, desde el principio recomiendo una estructura más cercana a producción:

```
src/  
├── config/  
├── controllers/  
├── models/  
├── routes/  
├── middlewares/  
├── app.js  
└── server.js
```

Porque en el siguiente módulo (MongoDB Atlas + Mongoose) ya comenzaremos a separar responsabilidades y evitar que todo quede dentro de `app.js`.

MÓDULO 3: MongoDB Atlas y Conexión con Mongoose

Objetivos de aprendizaje

- Crear una cuenta en MongoDB Atlas.
 - Crear un Cluster gratuito.
 - Configurar acceso a la base de datos.
 - Crear un usuario de base de datos.
 - Obtener la cadena de conexión.
 - Conectar Node.js con MongoDB Atlas usando Mongoose.
 - Comprender qué es un Schema.
 - Comprender qué es un Model.
-

1. ¿Qué es MongoDB Atlas?

MongoDB Atlas es el servicio cloud oficial de MongoDB. Nos permite tener la base de datos disponible desde Internet.

Arquitectura:

Render → MongoDB Atlas → Node.js Local

Cuando desplaguemos en Render, seguiremos utilizando la misma base de datos.

2. Crear Cuenta Atlas

Ingresar a: [MongoDB Atlas](#)

Puede utilizarse: Google, GitHub, Correo electrónico

3. Crear un Cluster

Una vez dentro:	Create Cluster
Seleccionar:	M0 Free Tier
Proveedor sugerido:	AWS
Región sugerida:	South America (São Paulo)

Crear Cluster. La creación puede tardar algunos minutos.

4. Configurar Network Access

MongoDB protege el acceso por IP.

Ir a: `Security` → `Network Access`

Agregar: `0.0.0.0/0`

Permite conexiones desde cualquier IP. Para ambientes productivos posteriormente restringiremos el acceso. Pero durante el curso simplifica el trabajo.

5. Crear Usuario de Base de Datos

Ir a: `Security` → `Database Access`

Crear usuario. Por ejemplo:

```
usuario: admin_inacap
password: *****
```

(Guarda la contraseña. La necesitaremos para conectarnos)

6. Obtener la Cadena de Conexión

Ir a: `Clusters` → `Connect`

Seleccionar: `Drivers`

Elegir: `Node.js`

Atlas entregará algo similar a:

```
mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

7. Crear Base de Datos del Proyecto

Nosotros utilizaremos: `inacap_academico`

La URI quedará similar a:

```
mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/inacap_academico?retryWrites=true&w=majority
```

8. Variables de Entorno

Archivo: `.env`

Agregar:

`PORT=3000`

`MONGODB_URI=mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/inacap_academico?retryWrites=true&w=majority`

9. Crear Carpeta Config

Estructura:

```
src/
├── config/
│   └── db.js
├── controllers/
├── middlewares/
├── models/
├── routes/
├── app.js
└── server.js
```

10. Crear Archivo db.js

Ruta: `src/config/db.js`

Contenido:

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("MongoDB conectado correctamente");
  } catch (error) {
    console.error("Error al conectar MongoDB");
    console.error(error.message);
    process.exit(1);
  }
};

export default connectDB;
```

Explicación detallada

```
const connectDB = async () => {}
```

La conexión tarda un tiempo. Por eso utilizamos `async`.

```
await mongoose.connect(...)
```

Espera hasta completar la conexión.

`try/catch`

Permite capturar errores.

```
process.exit(1)
```

Detiene la aplicación. No tiene sentido ejecutar una API si la base de datos no funciona.

11. Modificar server.js

```
import dotenv from "dotenv";
import app from "../app.js";
import connectDB from "../config/db.js";

dotenv.config();

const PORT = process.env.PORT || 3000;

const startServer = async () => {
  await connectDB();

  app.listen(PORT, () => {
    console.log(`Servidor ejecutándose en puerto ${PORT}`);
  });
};

startServer();
```

¿Por qué crear startServer()?

Porque queremos:

1. Conectar MongoDB.
2. Luego iniciar Express.

Servidor inicia → Carga .env → Conecta MongoDB → Inicia Express

12. Verificar Conexión

Ejecutar: `npm run dev`

Resultado esperado:

MongoDB conectado correctamente
Servidor ejecutándose en puerto 3000

13. ¿Qué es un Schema?

Un Schema define la estructura de los documentos.

Ejemplo: `nombre, apellido, correo, semestre`

El Schema ayuda a validar datos.

14. Crear Primer Modelo: Alumno

Ruta: `src/models/Alumno.js`

Contenido:

```
import mongoose from "mongoose";

const alumnoSchema = new mongoose.Schema(
  {
    rut: {
      type: String,
      required: true,
      unique: true
    },

    nombre: {
      type: String,
      required: true
    },

    apellido: {
      type: String,
      required: true
    },

    correo: {
      type: String,
      required: true
    },
  },
);
```

```
    semestre: {
      type: Number,
      required: true
    }
  },
  {
    timestamps: true
  }
);

const Alumno = mongoose.model("Alumno", alumnoSchema);

export default Alumno;
```

Resultado esperado del módulo

```
src/
├── config/
│   └── db.js
├── models/
│   └── Alumno.js
├── controllers/
├── routes/
├── middlewares/
├── app.js
└── server.js
```

MÓDULO 4: CRUD COMPLETO DE ALUMNOS

Objetivos de aprendizaje

- Comprender el patrón Controller.
 - Comprender el patrón Routes.
 - Crear endpoints REST.
 - Utilizar métodos de Mongoose.
 - Utilizar async/await.
 - Manejar errores.
 - Utilizar códigos HTTP correctamente.
 - Probar APIs con Thunder Client.
 - Implementar CRUD completo.
-

1. Arquitectura que utilizaremos

Hasta ahora:

```
src/  
├── config/  
├── models/  
├── app.js  
└── server.js
```

Ahora agregaremos:

```
src/  
├── config/  
├── controllers/  
│   └── alumnoController.js  
├── models/  
│   └── Alumno.js  
├── routes/  
│   └── alumnoRoutes.js  
├── app.js  
└── server.js
```

2. Crear controlador

Ruta: `src/controllers/alumnoController.js`

Importar modelo `import Alumno from "../models/Alumno.js";`

3. Primer endpoint: Obtener todos los alumnos

```
import Alumno from "../models/Alumno.js";

export const obtenerAlumnos = async (req, res) => {
  try {
    const alumnos = await Alumno.find();
    res.status(200).json(alumnos);
  } catch (error) {

    res.status(500).json({
      mensaje: "Error al obtener alumnos",
      error: error.message
    });
  }
};
```

Explicación línea por línea

Buscar todos `const alumnos = await Alumno.find();`

Equivalente conceptual: `SELECT * FROM alumnos`

Manejo de errores: `catch (error)`

Si MongoDB falla: `500 Internal Server Error`

4. Crear archivo de rutas

Ruta: `src/routes/alumnoRoutes.js`

```
import { Router } from "express";

const router = Router();

export default router;
```

¿Qué es Router?

Permite agrupar endpoints relacionados.

Ejemplo:

```
/api/alumnos  
/api/docentes  
/api/asignaturas
```

Cada recurso tendrá su propio archivo.

5. Conectar controlador

```
import { Router } from "express";  
import { obtenerAlumnos } from "../controllers/alumnoController.js";  
const router = Router();  
router.get("/", obtenerAlumnos);  
export default router;
```

Flujo completo

GET /api/alumnos → alumnoRoutes → obtenerAlumnos() → MongoDB

6. Registrar rutas en app.js

```
import express from "express";  
  
import alumnoRoutes from "../routes/alumnoRoutes.js";  
  
const app = express();  
  
app.use(express.json());  
  
app.use("/api/alumnos", alumnoRoutes);  
  
export default app;
```

Resultado

Solicitud: GET /api/alumnos

Respuesta: [] (Todavía no existen alumnos.)

7. Crear Alumno (POST)

Agregar al controlador:

```
export const crearAlumno = async (req, res) => {
  try {

    const alumno = await Alumno.create(req.body);

    res.status(201).json(alumno);

  } catch (error) {

    res.status(500).json({
      mensaje: "Error al crear alumno",
      error: error.message
    });

  }
};
```

¿Qué hace create()?

Inserta un documento.

`Alumno.create()`

Equivalente conceptual:

`INSERT INTO alumnos ...`

Registrar ruta

```
import { Router } from "express";

import {
  obtenerAlumnos,
  crearAlumno
} from "../controllers/alumnoController.js";

const router = Router();

router.get("/", obtenerAlumnos);
router.post("/", crearAlumno);

export default router;
```

Probar en Postman

POST http://localhost:3000/api/alumnos

Body

```
{
  "rut": "11111111-1",
  "nombre": "Juan",
  "apellido": "Pérez",
  "correo": "juan@inacapmail.cl",
  "semestre": 3
}
```

Respuesta

```
{
  "_id": "...",
  "rut": "11111111-1",
  "nombre": "Juan",
  "apellido": "Pérez",
  "correo": "juan@inacapmail.cl",
  "semestre": 3
}
```

8. Obtener alumno por ID

Agregar controlador:

```
export const obtenerAlumnoPorId = async (req, res) => {
  try {
    const alumno = await Alumno.findById(req.params.id);

    if (!alumno) {
      return res.status(404).json({
        mensaje: "Alumno no encontrado"
      });
    }

    res.status(200).json(alumno);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener alumno",
      error: error.message
    });
  }
};
```

9. Actualizar Alumno

Controlador:

```
export const actualizarAlumno = async (req, res) => {
  try {

    const alumno = await Alumno.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true
      }
    );

    if (!alumno) {
      return res.status(404).json({
        mensaje: "Alumno no encontrado"
      });
    }

    res.status(200).json(alumno);

  } catch (error) {

    res.status(500).json({
      mensaje: "Error al actualizar alumno",
      error: error.message
    });

  }
};
```

Explicación

new:true

Retorna documento actualizado. Sin esto: documento antiguo

runValidators:true

Vuelve a ejecutar validaciones del Schema. Muy importante.

10. Eliminar Alumno

Controlador:

```
export const eliminarAlumno = async (req, res) => {
  try {

    const alumno = await Alumno.findByIdAndDelete(
      req.params.id
    );

    if (!alumno) {
      return res.status(404).json({
        mensaje: "Alumno no encontrado"
      });
    }

    res.status(200).json({
      mensaje: "Alumno eliminado correctamente"
    });

  } catch (error) {

    res.status(500).json({
      mensaje: "Error al eliminar alumno",
      error: error.message
    });

  }
};
```

11. Archivo de rutas completo

```
import { Router } from "express";

import {
  obtenerAlumnos,
  obtenerAlumnoPorId,
  crearAlumno,
  actualizarAlumno,
  eliminarAlumno
} from "../controllers/alumnoController.js";

const router = Router();

router.get("/", obtenerAlumnos);
router.get("/:id", obtenerAlumnoPorId);
router.post("/", crearAlumno);
router.put("/:id", actualizarAlumno);
router.delete("/:id", eliminarAlumno);

export default router;
```

12. Mejorando validaciones

Actualmente `Alumno.create(req.body)` acepta cualquier correo.

Mejoraremos el Schema.

Alumno.js

```
correo: {  
  type: String,  
  required: true,  
  match: /^\S+@\S+\.\S+$/  
}
```

semestre

```
semestre: {  
  type: Number,  
  required: true,  
  min: 1,  
  max: 8  
}
```

13. Códigos HTTP utilizados

Operación	Código
GET	200
POST	201
PUT	200
DELETE	200
No encontrado	404
Error servidor	500

14. Primera refactorización profesional

En lugar de repetir: `catch (error)` en todos los métodos, más adelante crearemos un middleware global de errores.

Estructura al finalizar

```
src/  
├── config/  
│   └── db.js  
├── controllers/  
│   └── alumnoController.js  
├── models/  
│   └── Alumno.js  
├── routes/  
│   └── alumnoRoutes.js  
├── app.js  
└── server.js
```

MÓDULO 5: CRUD COMPLETO DE DOCENTES

1. El mismo patrón, un nuevo recurso

Ya conocemos el patrón completo gracias al módulo anterior: Modelo, Controlador y Rutas. Para Docentes repetiremos exactamente la misma receta, cambiando solamente los campos del Schema.

Alumno -> Modelo + Controlador + Rutas
Docente -> Modelo + Controlador + Rutas (mismo patrón)

2. Crear el modelo Docente

Ruta: `src/models/Docente.js`

```
import mongoose from "mongoose";

const docenteSchema = new mongoose.Schema(
  {
    rut: {
      type: String,
      required: true,
      unique: true
    },
    nombre: {
      type: String,
      required: true
    },
    apellido: {
      type: String,
      required: true
    },
    correo: {
      type: String,
      required: true,
      match: /^\S+@\S+\.\S+$/
    },
    especialidad: {
      type: String,
      required: true
    }
  },
  {
    timestamps: true
  }
);

const Docente = mongoose.model("Docente", docenteSchema);

export default Docente;
```

3. Crear el controlador de Docentes

Ruta: `src/controllers/docenteController.js`

```
import Docente from "../models/Docente.js";

export const obtenerDocentes = async (req, res) => {
  try {
    const docentes = await Docente.find();
    res.status(200).json(docentes);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener docentes",
      error: error.message
    });
  }
};

export const obtenerDocentePorId = async (req, res) => {
  try {
    const docente = await Docente.findById(req.params.id);

    if (!docente) {
      return res.status(404).json({
        mensaje: "Docente no encontrado"
      });
    }

    res.status(200).json(docente);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener docente",
      error: error.message
    });
  }
};

export const crearDocente = async (req, res) => {
  try {
    const docente = await Docente.create(req.body);
    res.status(201).json(docente);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al crear docente",
      error: error.message
    });
  }
};
```

```

export const actualizarDocente = async (req, res) => {
  try {
    const docente = await Docente.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!docente) {
      return res.status(404).json({
        mensaje: "Docente no encontrado"
      });
    }

    res.status(200).json(docente);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al actualizar docente",
      error: error.message
    });
  }
};

export const eliminarDocente = async (req, res) => {
  try {
    const docente = await Docente.findByIdAndDelete(req.params.id);

    if (!docente) {
      return res.status(404).json({
        mensaje: "Docente no encontrado"
      });
    }

    res.status(200).json({
      mensaje: "Docente eliminado correctamente"
    });
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al eliminar docente",
      error: error.message
    });
  }
};

```

4. Crear las rutas de Docentes

Ruta: `src/routes/docenteRoutes.js`

```
import { Router } from "express";
import {
  obtenerDocentes,
  obtenerDocentePorId,
  crearDocente,
  actualizarDocente,
  eliminarDocente
} from "../controllers/docenteController.js";

const router = Router();

router.get("/", obtenerDocentes);
router.get("/:id", obtenerDocentePorId);
router.post("/", crearDocente);
router.put("/:id", actualizarDocente);
router.delete("/:id", eliminarDocente);

export default router;
```

5. Registrar la ruta en app.js

```
import express from "express";
import alumnoRoutes from "../routes/alumnoRoutes.js";
import docenteRoutes from "../routes/docenteRoutes.js";

const app = express();

app.use(express.json());

app.use("/api/alumnos", alumnoRoutes);
app.use("/api/docentes", docenteRoutes);

export default app;
```

6. Probar en Postman / Thunder Client

Repetimos las mismas pruebas realizadas con Alumnos, ahora apuntando a `/api/docentes`:

```
POST http://localhost:3000/api/docentes
GET http://localhost:3000/api/docentes
GET http://localhost:3000/api/docentes/:id
PUT http://localhost:3000/api/docentes/:id
DELETE http://localhost:3000/api/docentes/:id
```

Body de ejemplo para crear un docente:

```
{
  "rut": "99999999-9",
  "nombre": "Carlos",
  "apellido": "Gómez",
  "correo": "carlos@inacap.cl",
  "especialidad": "Backend"
}
```

Estructura del proyecto al finalizar el módulo

```
src/
├── config/
│   └── db.js
├── controllers/
│   ├── alumnoController.js
│   └── docenteController.js
├── models/
│   ├── Alumno.js
│   └── Docente.js
├── routes/
│   ├── alumnoRoutes.js
│   └── docenteRoutes.js
├── app.js
└── server.js
```

MÓDULO 6: CRUD DE ASIGNATURAS Y RELACIONES CON OBJECTID

1. ¿Cómo se relacionan los datos en MongoDB?

MongoDB es una base de datos no relacional, pero eso no significa que no podamos relacionar información. La forma más común de hacerlo es guardando el `_id` de un documento dentro de otro documento.

```
Asignatura
{
  "codigo": "PGY4121",
  "nombre": "Programación Backend",
  "docente": "6512f1a2b3c4d5e6f7a8b9c0"  <- ObjectId del docente
}
```

2. Crear el modelo Asignatura

Ruta: `src/models/Asignatura.js`

```
import mongoose from "mongoose";
const asignaturaSchema = new mongoose.Schema(
  {
    codigo: {
      type: String,
      required: true,
      unique: true
    },
    nombre: {
      type: String,
      required: true
    },
    semestre: {
      type: Number,
      required: true,
      min: 1,
      max: 8
    },
    docente: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Docente",
      required: true
    }
  },
  {
    timestamps: true
  }
);
const Asignatura = mongoose.model("Asignatura", asignaturaSchema);
export default Asignatura;
```

El campo `docente` es de tipo `ObjectId` y la propiedad `ref: "Docente"` le indica a Mongoose a qué colección pertenece esa referencia. Gracias a esto luego podremos usar `populate()`.

3. Crear el controlador de Asignaturas

Ruta: `src/controllers/asignaturaController.js`

```
import Asignatura from "../../models/Asignatura.js";

export const obtenerAsignaturas = async (req, res) => {
  try {
    const asignaturas = await Asignatura.find()
      .populate("docente", "nombre apellido especialidad");

    res.status(200).json(asignaturas);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener asignaturas",
      error: error.message
    });
  }
};

export const obtenerAsignaturaPorId = async (req, res) => {
  try {
    const asignatura = await Asignatura.findById(req.params.id)
      .populate("docente", "nombre apellido especialidad");

    if (!asignatura) {
      return res.status(404).json({
        mensaje: "Asignatura no encontrada"
      });
    }

    res.status(200).json(asignatura);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener asignatura",
      error: error.message
    });
  }
};

export const crearAsignatura = async (req, res) => {
  try {
    const asignatura = await Asignatura.create(req.body);
    res.status(201).json(asignatura);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al crear asignatura",
      error: error.message
    });
  }
};
```

```

export const actualizarAsignatura = async (req, res) => {
  try {
    const asignatura = await Asignatura.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!asignatura) {
      return res.status(404).json({
        mensaje: "Asignatura no encontrada"
      });
    }

    res.status(200).json(asignatura);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al actualizar asignatura",
      error: error.message
    });
  }
};

export const eliminarAsignatura = async (req, res) => {
  try {
    const asignatura = await Asignatura.findByIdAndDelete(req.params.id);

    if (!asignatura) {
      return res.status(404).json({
        mensaje: "Asignatura no encontrada"
      });
    }

    res.status(200).json({
      mensaje: "Asignatura eliminada correctamente"
    });
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al eliminar asignatura",
      error: error.message
    });
  }
};

```

4. ¿Qué hace populate()?

Sin `populate()`, el campo `docente` solo mostraría el `ObjectId`:

```
{
  "codigo": "PGY4121",
  "nombre": "Programación Backend",
  "docente": "6512f1a2b3c4d5e6f7a8b9c0"
}
```

Con `populate("docente", "nombre apellido especialidad")`, Mongoose reemplaza ese `ObjectId` por el documento completo (o solo los campos indicados):

```
{
  "codigo": "PGY4121",
  "nombre": "Programación Backend",
  "docente": {
    "_id": "6512f1a2b3c4d5e6f7a8b9c0",
    "nombre": "Carlos",
    "apellido": "Gómez",
    "especialidad": "Backend"
  }
}
```

`populate()` ejecuta internamente una segunda consulta a la colección `Docentes` y combina los resultados. Es la forma de Mongoose de simular un JOIN.

5. Crear las rutas de Asignaturas

Ruta: `src/routes/asignaturaRoutes.js`

```
import { Router } from "express";
import {
  obtenerAsignaturas,
  obtenerAsignaturaPorId,
  crearAsignatura,
  actualizarAsignatura,
  eliminarAsignatura
} from "../controllers/asignaturaController.js";

const router = Router();

router.get("/", obtenerAsignaturas);
router.get("/:id", obtenerAsignaturaPorId);
router.post("/", crearAsignatura);
router.put("/:id", actualizarAsignatura);
router.delete("/:id", eliminarAsignatura);

export default router;
```

6. Registrar la ruta en app.js

```
import express from "express";
import alumnoRoutes from "../routes/alumnoRoutes.js";
import docenteRoutes from "../routes/docenteRoutes.js";
import asignaturaRoutes from "../routes/asignaturaRoutes.js";
const app = express();
app.use(express.json());
app.use("/api/alumnos", alumnoRoutes);
app.use("/api/docentes", docenteRoutes);
app.use("/api/asignaturas", asignaturaRoutes);
export default app;
```

7. Probar la relación en Postman

Paso 1: crear un docente y copiar su `_id` desde la respuesta.

Paso 2: crear una asignatura usando ese `_id` en el campo docente:

POST `http://localhost:3000/api/asignaturas`

```
{  "codigo": "PGY4121",  "nombre": "Programación Backend",
  "semestre": 4,  "docente": "6512f1a2b3c4d5e6f7a8b9c0" }
```

Paso 3: hacer GET `/api/asignaturas` y observar que el campo docente viene con la información completa gracias a `populate()`.

Estructura del proyecto al finalizar el módulo

```
src/
├── config/
│   └── db.js
├── controllers/
│   ├── alumnoController.js
│   ├── docenteController.js
│   └── asignaturaController.js
├── models/
│   ├── Alumno.js
│   ├── Docente.js
│   └── Asignatura.js
├── routes/
│   ├── alumnoRoutes.js
│   ├── docenteRoutes.js
│   └── asignaturaRoutes.js
├── app.js
└── server.js
```

MÓDULO 7: CONSULTAS AVANZADAS CON AGGREGATE Y LOOKUP

1. ¿Qué es aggregate()?

`find()` sirve para buscar documentos. `aggregate()` sirve para procesar y transformar datos: agrupar, contar, sumar, combinar colecciones, etc. Funciona mediante un pipeline: una lista de etapas que los documentos van atravesando, una tras otra.

Colección -> Etapa 1 -> Etapa 2 -> Etapa 3 -> Resultado

2. Etapa \$match

Filtra documentos, igual que el `find()`. `{ $match: { semestre: 4 } }`

3. Etapa \$group

Agrupar documentos según un campo y permite calcular totales.

```
{ $group: {  
  _id: "$semestre",  
  totalAlumnos: { $sum: 1 } } }
```

`_id` define la clave de agrupación (en este caso, el semestre). `$sum: 1` suma 1 por cada documento, es decir, cuenta cuántos hay en cada grupo.

4. Ejemplo completo: alumnos por semestre

Agregar al controlador `alumnoController.js`:

```
export const alumnosPorSemestre = async (req, res) => {  
  try {  
    const resultado = await Alumno.aggregate([  
      { $group: {  
        _id: "$semestre",  
        totalAlumnos: { $sum: 1 } } },  
      {  
        $sort: { _id: 1 }  
      }  
    ]);  
    res.status(200).json(resultado);  
  } catch (error) {  
    res.status(500).json({  
      mensaje: "Error al obtener estadísticas",  
      error: error.message  
    });  
  }  
};
```

Registrar la ruta en `alumnoRoutes.js`:

```
router.get("/estadisticas/por-semester", alumnosPorSemestre);
```

Respuesta esperada:

```
[
  { "_id": 3, "totalAlumnos": 5 },
  { "_id": 4, "totalAlumnos": 8 }
]
```

5. ¿Qué es \$lookup?

MongoDB no tiene JOIN como las bases de datos relacionales, pero `$lookup` permite combinar información de otra colección dentro del mismo pipeline, de forma similar a un JOIN.

```
{
  $lookup: {
    from: "docentes",
    localField: "docente",
    foreignField: "_id",
    as: "informacionDocente"
  }
}
```

Propiedad	Significado
from	Colección con la que se combina (docentes)
localField	Campo de la colección actual (docente)
foreignField	Campo de la colección externa (_id)
as	Nombre del nuevo arreglo con el resultado

6. Endpoint de Asignaturas con \$lookup

Agregar al controlador `asignaturaController.js`:

```
export const asignaturasConDocente = async (req, res) => {
  try {
    const resultado = await Asignatura.aggregate([
      {
        $lookup: {
          from: "docentes",
          localField: "docente",
          foreignField: "_id",
          as: "informacionDocente"
        }
      },
      {
        $unwind: "$informacionDocente"
      },
      {
        $project: {
          codigo: 1,
          nombre: 1,
          semestre: 1,
          "informacionDocente.nombre": 1,
          "informacionDocente.apellido": 1,
          "informacionDocente.especialidad": 1
        }
      }
    ]);

    res.status(200).json(resultado);
  } catch (error) {
    res.status(500).json({
      mensaje: "Error al obtener asignaturas con docente",
      error: error.message
    });
  }
};
```

`$unwind` convierte el arreglo `informacionDocente` (que `$lookup` siempre devuelve como arreglo) en un objeto simple, ya que en este caso cada asignatura tiene un único docente. `$project` selecciona qué campos queremos mostrar en la respuesta final.

Registrar la ruta en `asignaturaRoutes.js`:

```
router.get("/reportes/con-docente", asignaturasConDocente);
```

7. populate() versus aggregate() + \$lookup

Característica	populate()	aggregate() + \$lookup
Facilidad de uso	Muy simple	Requiere más código
Rendimiento en grandes volúmenes	Menor	Mayor
Permite agrupar y calcular totales	No	Sí
Uso recomendado	Relaciones simples	Reportes y estadísticas

MÓDULO 8: MIDDLEWARE GLOBAL DE ERRORES Y CORS

1. El problema actual

Hasta ahora, cada método del controlador repite el mismo bloque `try/catch`. Esto funciona, pero duplica código y dificulta mantener un formato de error consistente en toda la API.

```
try {  
  // lógica  
} catch (error) {  
  res.status(500).json({ mensaje: '...', error: error.message });  
}
```

La solución profesional es delegar el error a Express mediante `next(error)`, y manejarlo en un único middleware global.

2. ¿Qué es un middleware?

Un middleware es una función que se ejecuta entre la solicitud y la respuesta. Recibe tres parámetros: `req`, `res` y `next`. Un middleware de errores recibe un cuarto parámetro adicional al inicio: `error`.

```
(req, res, next) => { ... }           // middleware normal  
(error, req, res, next) => { ... }    // middleware de errores
```

3. Crear el middleware de errores

Ruta: `src/middlewares/errorHandler.js`

```
export const errorHandler = (error, req, res, next) => {  
  console.error(error.stack);  
  
  const statusCode = error.statusCode || 500;  
  
  res.status(statusCode).json({  
    mensaje: error.mensaje || "Error interno del servidor",  
    error: error.message  
  });  
};
```

4. Middleware para rutas no encontradas

Ruta: src/middlewares/notFound.js

```
export const notFound = (req, res, next) => {
  res.status(404).json({
    mensaje: `Ruta no encontrada: ${req.originalUrl}`
  });
};
```

5. Refactorizar el controlador de Alumnos

Reemplazamos cada `catch` para que delegue el error con `next(error)` en lugar de responder directamente:

```
import Alumno from "../models/Alumno.js";

export const obtenerAlumnos = async (req, res, next) => {
  try {
    const alumnos = await Alumno.find();
    res.status(200).json(alumnos);
  } catch (error) {
    next(error);
  }
};

export const crearAlumno = async (req, res, next) => {
  try {
    const alumno = await Alumno.create(req.body);
    res.status(201).json(alumno);
  } catch (error) {
    next(error);
  }
};

// El mismo patrón se aplica a obtenerAlumnoPorId,
// actualizarAlumno y eliminarAlumno.
```

El mismo cambio se aplica de forma idéntica a los controladores de Docentes y Asignaturas.

6. Registrar los middlewares en app.js

El orden es muy importante: los middlewares de error y de ruta no encontrada deben registrarse después de todas las rutas.

```
import express from "express";
import alumnoRoutes from "../routes/alumnoRoutes.js";
import docenteRoutes from "../routes/docenteRoutes.js";
import asignaturaRoutes from "../routes/asignaturaRoutes.js";
import { errorHandler } from "../middlewares/errorHandler.js";
import { notFound } from "../middlewares/notFound.js";

const app = express();

app.use(express.json());

app.use("/api/alumnos", alumnoRoutes);
app.use("/api/docentes", docenteRoutes);
app.use("/api/asignaturas", asignaturaRoutes);

app.use(notFound);
app.use(errorHandler);

export default app;
```

7. ¿Qué es CORS?

CORS significa Cross-Origin Resource Sharing. Por seguridad, los navegadores bloquean por defecto las solicitudes que un sitio web hace hacia un dominio distinto al propio.

Nuestro backend vivirá en un dominio (por ejemplo, Render) y el frontend en otro (por ejemplo, Vercel). Sin CORS, el navegador bloquearía esas solicitudes.

```
Frontend (https://miapp.vercel.app)
  |
  |  Solicitud HTTP
  v
Backend (https://miapp.onrender.com)  <- dominio distinto, requiere CORS
```

8. Configurar CORS en Express

Ya instalamos la dependencia `cors` en el Módulo 2. Ahora la utilizamos:

```
import express from "express";
import cors from "cors";
import alumnoRoutes from "../routes/alumnoRoutes.js";
import docenteRoutes from "../routes/docenteRoutes.js";
import asignaturaRoutes from "../routes/asignaturaRoutes.js";
import { errorHandler } from "../middlewares/errorHandler.js";
import { notFound } from "../middlewares/notFound.js";

const app = express();

app.use(cors());
app.use(express.json());

app.use("/api/alumnos", alumnoRoutes);
app.use("/api/docentes", docenteRoutes);
app.use("/api/asignaturas", asignaturaRoutes);

app.use(notFound);
app.use(errorHandler);

export default app;
```

`cors()` sin parámetros permite solicitudes desde cualquier origen. Es válido durante el desarrollo, pero en producción es recomendable restringirlo a la URL real del frontend.

9. Restringir CORS mediante variables de entorno

Agregar en `.env`:

```
FRONTEND_URL=http://localhost:5173
```

Y configurar `cors()` con ese origen:

```
import dotenv from "dotenv";
dotenv.config();

app.use(
  cors({
    origin: process.env.FRONTEND_URL
  })
);
```

Cuando despliéguenos en producción, esta variable se actualizará con la URL real del frontend en Vercel.

Estructura del proyecto al finalizar el módulo

```
src/
├── config/
│   └── db.js
├── controllers/
│   ├── alumnoController.js
│   ├── docenteController.js
│   └── asignaturaController.js
├── middlewares/
│   ├── errorHandler.js
│   └── notFound.js
├── models/
│   ├── Alumno.js
│   ├── Docente.js
│   └── Asignatura.js
├── routes/
│   ├── alumnoRoutes.js
│   ├── docenteRoutes.js
│   └── asignaturaRoutes.js
├── app.js
└── server.js
```

Con este módulo, el backend queda completo y listo para producción. A continuación corresponde desplegarlo.

MÓDULO 9: DEPLOY DEL BACKEND EN RENDER

1. Revisar el package.json

Render necesita saber cómo instalar e iniciar el proyecto. Confirmamos que existan los scripts definidos desde el Módulo 2:

```
{
  "name": "sistema-academico-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  }
}
```

Render utilizará `npm start` para iniciar el servidor en producción, por lo que el script `start` es obligatorio.

2. Confirmar el archivo .gitignore

```
node_modules
.env
```

Nunca subimos `node_modules` ni `.env` a GitHub. Las variables de entorno se configurarán directamente en Render.

3. Subir el proyecto a GitHub

```
git init
git add .
git commit -m "Backend Sistema Académico"
git branch -M main
git remote add origin https://github.com/usuario/sistema-academico-api.git
git push -u origin main
```

4. Crear cuenta en Render

Ingresar a Render (<https://render.com>) y crear una cuenta. Puede iniciarse sesión directamente con la cuenta de GitHub para simplificar la conexión del repositorio.

5. Crear un Web Service

- Dashboard de Render → New → Web Service.
- Conectar el repositorio de GitHub del backend.
- Seleccionar la rama main.

6. Configuración del servicio

Campo	Valor
Name	sistema-academico-api
Region	Cercana a los usuarios (ej. Oregon / São Paulo según disponibilidad)
Branch	main
Build Command	npm install
Start Command	npm start
Instance Type	Free

7. Configurar variables de entorno en Render

En la sección Environment del servicio, agregar las mismas variables que existen en el `.env` local:

```
PORT=10000
MONGODB_URI=mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/inacap_academico?retryWrites=true&w=majority
FRONTEND_URL=http://localhost:5173
```

Render asigna su propio puerto mediante la variable `PORT`; nuestro `server.js` ya está preparado para usarla porque utilizamos `process.env.PORT || 3000`.

8. Verificar acceso desde MongoDB Atlas

En el Módulo 3 configuramos Network Access con `0.0.0.0/0`, lo que permite que Render (y cualquier otro origen) se conecte a la base de datos sin restricciones adicionales.

9. Deploy

Al confirmar la creación del servicio, Render ejecuta automáticamente:

1. `git clone` del repositorio
2. `npm install`
3. `npm start`

El proceso puede tardar algunos minutos. Al finalizar, Render entrega una URL pública similar a:

`https://sistema-academico-api.onrender.com`

10. Probar la API en producción

Con Postman o Thunder Client, repetimos las pruebas anteriores reemplazando `localhost:3000` por la URL de Render:

```
GET https://sistema-academico-api.onrender.com/api/alumnos
GET https://sistema-academico-api.onrender.com/api/docentes
GET https://sistema-academico-api.onrender.com/api/asignaturas
```

Nota: en el plan gratuito, Render "duerme" el servicio tras un periodo de inactividad. La primera solicitud luego de ese período puede demorar algunos segundos mientras el servicio despierta.

Arquitectura de despliegue actual

```
GitHub (código)
  |
  v
Render (backend en producción) <-----> MongoDB Atlas (base de datos)
```

MÓDULO 10: FRONTEND CON REACT

1. Crear el proyecto con Vite

Vite es la herramienta recomendada actualmente para crear proyectos React: es más rápida que Create React App y es el estándar utilizado en la industria.

```
npm create vite@latest sistema-academico-web -- --template react
cd sistema-academico-web
npm install
```

2. Instalar dependencias del frontend

```
npm install axios react-router-dom
```

Paquete	Para qué sirve
axios	Realizar solicitudes HTTP hacia el backend
react-router-dom	Manejar navegación entre páginas (rutas del frontend)

3. Estructura del proyecto frontend

```
sistema-academico-web/
├── src/
│   ├── api/
│   │   └── axios.js
│   ├── components/
│   │   └── AlumnoForm.jsx
│   ├── pages/
│   │   ├── ListadoAlumnos.jsx
│   │   ├── CrearAlumno.jsx
│   │   └── EditarAlumno.jsx
│   ├── App.jsx
│   └── main.jsx
├── .env
├── package.json
└── vite.config.js
```

4. Variables de entorno del frontend

Crear archivo `.env` en la raíz del proyecto frontend. En Vite, las variables deben comenzar con el prefijo `VITE_`:

```
VITE_API_URL=http://localhost:3000/api
```

Cuando despliegamos en Vercel, esta variable se reemplazará con la URL real del backend en Render.

5. Crear la instancia de Axios

Ruta: `src/api/axios.js`

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL
});

export default api;
```

Centralizar la configuración de Axios en un único archivo evita repetir la URL base en cada componente.

6. Listado de Alumnos (Read)

Ruta: `src/pages/ListadoAlumnos.jsx`

```
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import api from "../api/axios.js";

function ListadoAlumnos() {
  const [alumnos, setAlumnos] = useState([]);
  const [cargando, setCargando] = useState(true);

  const obtenerAlumnos = async () => {
    try {
      const respuesta = await api.get("/alumnos");
      setAlumnos(respuesta.data);
    } catch (error) {
      console.error("Error al obtener alumnos", error);
    } finally {
      setCargando(false);
    }
  };

  useEffect(() => {
    obtenerAlumnos();
  }, []);

  const eliminarAlumno = async (id) => {
    const confirmar = window.confirm("¿Eliminar este alumno?");
    if (!confirmar) return;

    try {
      await api.delete(`/alumnos/${id}`);
      setAlumnos(alumnos.filter((alumno) => alumno._id !== id));
    } catch (error) {
      console.error("Error al eliminar alumno", error);
    }
  };

  if (cargando) return <p>Cargando alumnos...</p>;

  return (
    <div>
      <h1>Listado de Alumnos</h1>
      <Link to="/alumnos/nuevo">Agregar alumno</Link>

      <table>
        <thead>
          <tr>
            <th>Nombre</th>
            <th>Apellido</th>
            <th>Correo</th>
            <th>Semestre</th>
            <th>Acciones</th>
          </tr>
        </thead>
      </table>
    </div>
  );
}
```

```

    </thead>
    <tbody>
      {alumnos.map((alumno) => (
        <tr key={alumno._id}>
          <td>{alumno.nombre}</td>
          <td>{alumno.apellido}</td>
          <td>{alumno.correo}</td>
          <td>{alumno.semestre}</td>
          <td>
            <Link to={`/alumnos/editar/${alumno._id}`}>Editar</Link>
            <button onClick={() => eliminarAlumno(alumno._id)}>
              Eliminar
            </button>
          </td>
        </tr>
      ) )}
    </tbody>
  </table>
</div>
);
}

```

```
export default ListadoAlumnos;
```

useEffect **ejecuta** obtenerAlumnos() apenas el componente se monta, simulando el mismo momento en que, en Postman, ejecutábamos manualmente GET /api/alumnos.

7. Formulario reutilizable de Alumno

Ruta: `src/components/AlumnoForm.jsx`.

Este formulario se reutilizará tanto para crear como para editar un alumno.

```
import { useState } from "react";

function AlumnoForm({ alumnoInicial, onGuardar }) {
  const [form, setForm] = useState(
    alumnoInicial || {
      rut: "",
      nombre: "",
      apellido: "",
      correo: "",
      semestre: 1
    }
  );

  const manejarCambio = (evento) => {
    setForm({
      ...form,
      [evento.target.name]: evento.target.value
    });
  };

  const manejarEnvio = (evento) => {
    evento.preventDefault();
    onGuardar(form);
  };

  return (
    <form onSubmit={manejarEnvio}>
      <input
        name="rut"
        placeholder="RUT"
        value={form.rut}
        onChange={manejarCambio}
      />
      <input
        name="nombre"
        placeholder="Nombre"
        value={form.nombre}
        onChange={manejarCambio}
      />
      <input
        name="apellido"
        placeholder="Apellido"
        value={form.apellido}
        onChange={manejarCambio}
      />
      <input
        name="correo"
        placeholder="Correo"
        value={form.correo}

```

```
        onChange={manejarCambio}
      />
      <input
        name="semestre"
        type="number"
        value={form.semestre}
        onChange={manejarCambio}
      />
      <button type="submit">Guardar</button>
    </form>
  );
}

export default AlumnoForm;
```

8. Crear Alumno (Create)

Ruta: `src/pages/CrearAlumno.jsx`

```
import { useNavigate } from "react-router-dom";
import api from "../../api/axios.js";
import AlumnoForm from "../../components/AlumnoForm.jsx";

function CrearAlumno() {
  const navegar = useNavigate();

  const guardarAlumno = async (datos) => {
    try {
      await api.post("/alumnos", datos);
      navegar("/");
    } catch (error) {
      console.error("Error al crear alumno", error);
    }
  };

  return (
    <div>
      <h1>Agregar Alumno</h1>
      <AlumnoForm onGuardar={guardarAlumno} />
    </div>
  );
}

export default CrearAlumno;
```

9. Editar Alumno (Update)

Ruta: `src/pages/EditarAlumno.jsx`

```
import { useEffect, useState } from "react";
import { useNavigate, useParams } from "react-router-dom";
import api from "../api/axios.js";
import AlumnoForm from "../components/AlumnoForm.jsx";

function EditarAlumno() {
  const { id } = useParams();
  const navegar = useNavigate();
  const [alumno, setAlumno] = useState(null);

  useEffect(() => {
    const obtenerAlumno = async () => {
      const respuesta = await api.get(`/alumnos/${id}`);
      setAlumno(respuesta.data);
    };

    obtenerAlumno();
  }, [id]);

  const actualizarAlumno = async (datos) => {
    try {
      await api.put(`/alumnos/${id}`, datos);
      navegar("/");
    } catch (error) {
      console.error("Error al actualizar alumno", error);
    }
  };

  if (!alumno) return <p>Cargando...</p>;

  return (
    <div>
      <h1>Editar Alumno</h1>
      <AlumnoForm alumnoInicial={alumno} onGuardar={actualizarAlumno} />
    </div>
  );
}

export default EditarAlumno;
```

10. Configurar React Router

Ruta:

src/App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ListadoAlumnos from "../pages/ListadoAlumnos.jsx";
import CrearAlumno from "../pages/CrearAlumno.jsx";
import EditarAlumno from "../pages/EditarAlumno.jsx";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<ListadoAlumnos />} />
        <Route path="/alumnos/nuevo" element={<CrearAlumno />} />
        <Route path="/alumnos/editar/:id" element={<EditarAlumno />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

`BrowserRouter` habilita la navegación, `Routes` agrupa las rutas y cada `Route` asocia una URL con el componente que debe mostrarse.

11. Ejecutar el frontend en desarrollo

```
npm run dev
```

Vite levantará el proyecto, normalmente en `http://localhost:5173`. Con el backend corriendo en el puerto 3000 (`npm run dev` dentro de la carpeta del backend), el sistema completo ya queda funcionando de forma local: React consulta Express, Express consulta MongoDB Atlas.

12. Actividad práctica

Replicar el mismo patrón (listado, crear, editar, eliminar) para las colecciones Docentes y Asignaturas, reutilizando `api.js` y el mismo enfoque de componentes.

MÓDULO 11: DEPLOY DEL FRONTEND EN VERCEL

1. Confirmar el archivo .gitignore

```
node_modules
dist
.env
```

2. Subir el frontend a GitHub

```
git init
git add .
git commit -m "Frontend Sistema Académico"
git branch -M main
git remote add origin https://github.com/usuario/sistema-academico-web.git
git push -u origin main
```

3. Crear cuenta en Vercel

Ingresa a Vercel (<https://vercel.com>) e inicia sesión con la cuenta de GitHub, lo que permite importar repositorios directamente.

4. Importar el proyecto

- Dashboard de Vercel → Add New → Project.
- Seleccionar el repositorio sistema-academico-web.
- Vercel detecta automáticamente que es un proyecto Vite + React.

5. Configuración del build

Campo	Valor
Framework Preset	Vite
Build Command	npm run build
Output Directory	dist
Install Command	npm install

Estos valores son detectados automáticamente por Vercel al reconocer un proyecto Vite, por lo que normalmente no es necesario modificarlos.

6. Variables de entorno en Vercel

En la sección Environment Variables del proyecto, agregar la URL real del backend desplegado en Render:

```
VITE_API_URL=https://sistema-academico-api.onrender.com/api
```

7. Deploy

Al confirmar, Vercel ejecuta `npm install` y `npm run build`, y entrega una URL pública similar a:

```
https://sistema-academico-web.vercel.app
```

8. Actualizar CORS en el backend

Ahora que conocemos la URL final del frontend, actualizamos la variable de entorno `FRONTEND_URL` en Render para reemplazar el valor de desarrollo (`http://localhost:5173`):

```
FRONTEND_URL=https://sistema-academico-web.vercel.app
```

Tras guardar la variable, Render reinicia automáticamente el servicio para aplicar el cambio.

9. Verificación final de extremo a extremo

- Abrir la URL del frontend en Vercel.
- Verificar que el listado de alumnos cargue datos reales desde MongoDB Atlas.
- Crear un alumno desde el formulario y confirmar que aparece en el listado.
- Editar y eliminar un alumno desde la interfaz.

Arquitectura final del proyecto

```
Usuario
|
v
Frontend React -> Vercel (https://sistema-academico-web.vercel.app)
|
| HTTP / Axios
v
Backend Express -> Render (https://sistema-academico-api.onrender.com)
|
| Mongoose
v
MongoDB Atlas (base de datos en la nube)
```

Cierre del curso

Con este módulo concluye el desarrollo del Sistema Académico simplificado. A lo largo del curso se construyó una aplicación full stack completa y funcional, siguiendo una progresión pensada para el aprendizaje:

- Fundamentos de APIs REST, HTTP y arquitectura cliente-servidor.
- Creación de un servidor Express y configuración de variables de entorno.
- Conexión a MongoDB Atlas mediante Mongoose y modelado de datos.
- CRUD completo de Alumnos, Docentes y Asignaturas.
- Relaciones entre colecciones mediante ObjectId y populate().
- Consultas avanzadas con aggregate() y \$lookup.
- Middleware global de errores y configuración de CORS.
- Deploy del backend en Render.
- Construcción del frontend en React con Axios y React Router.
- Deploy del frontend en Vercel.

El proyecto resultante queda 100% funcional y desplegado en producción, replicando el flujo de trabajo real utilizado por equipos de desarrollo full stack: backend, base de datos, frontend y despliegue en la nube.

Posibles extensiones del proyecto

- Autenticación de usuarios con JWT.
- Roles de acceso (administrador, docente, alumno).
- Paginación y búsqueda en los listados.
- Subida de archivos (por ejemplo, fotografía del alumno).
- Pruebas automatizadas con Jest o Supertest.
- Documentación de la API con Swagger.